

SENIOR SWE INTERVIEW SERIES

Module 2: TCP

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 2 — TCP (Highest Priority)

TCP is the single most-asked networking area in senior backend interviews. Expect packet-level follow-ups: interviewers at Google/Meta/Amazon will push until you draw sequence numbers.

Topic 2.1 — Three-Way Handshake

1. Why Interviewers Ask This

It's the canonical filter question. Weak candidates recite “SYN, SYN-ACK, ACK”; strong candidates explain *what state each side allocates, what can go wrong, and what data rides in each packet*. Follow-ups (SYN floods, TFO, simultaneous open) separate levels.

2. Core Concept

Three packets establish two things: (a) both directions are alive, and (b) both sides exchange initial sequence numbers (ISNs) so the byte-stream numbering can start. Options (MSS, window scale, SACK-permitted, timestamps) are exchanged *only here*.

3. Internal Working

- Client: `connect()` → kernel picks ephemeral port + random ISN (randomized to prevent off-path injection) → sends SYN → state `SYN_SENT`.
- Server: SYN arrives → entry in **SYN queue** (half-open), replies SYN-ACK → state `SYN_RECV`. With **SYN cookies**, the server encodes state into its ISN and stores nothing.
- Client ACKs → server moves connection to **accept queue** → app's `accept()` pops it. Both `ESTABLISHED`.

4. Packet Flow Explanation

```
1. C→S SYN,      seq=x,      options: MSS 1460, WS 7, SACK-OK, TS
2. S→C SYN-ACK,  seq=y, ack=x+1, options: MSS 1460, WS 7, SACK-OK, TS
3. C→S ACK,      seq=x+1, ack=y+1 (can already carry data)
```

SYN and FIN each consume one sequence number (that's why `ack=x+1` with no data). Cost: exactly 1 RTT before the client can send data (0 RTT with TCP Fast Open, which carries data on the SYN using a cookie from a prior connection).

5. ASCII Diagram

```
CLIENT                                SERVER
SYN_SENT  --- SYN seq=x -----> (SYN queue) SYN_RECV
ESTABLISHED <-- SYN-ACK y, ack=x+1 --
          --- ACK ack=y+1 -----> (accept queue) ESTABLISHED
                                   app accept() -> fd
Timeline cost: 1 RTT (+1 more for TLS 1.3, +2 for TLS 1.2)
```

6. Real Production Example

Every connection-pool design decision flows from this 1-RTT cost: databases (pgbouncer), HTTP keep-alive, gRPC channels all exist to amortize handshakes. Cloudflare mitigates SYN floods for customers using SYN cookies at the edge — millions of spoofed SYNs consume zero server memory.

7. Advantages

- Guarantees both directions work before data flows; protects against stale/duplicated SYNs from old connections (ISN + sequence validation).

8. Trade-offs

- 1 RTT of latency per new connection — brutal for short requests over 100ms+ paths.
- Half-open state is a DoS vector (SYN flood) unless cookies are used.
- SYN cookies discard options (limited MSS encoding, no window scale in classic form) — a real trade-off worth naming.

9. Common Mistakes

- Saying the handshake “negotiates the connection speed” — it doesn’t; congestion control starts after.
- Not knowing SYN queue vs accept queue are different (and overflow differently).
- Forgetting data can ride on the third ACK.

10. Performance Impact

For a 5ms-RTT datacenter call the handshake is noise; for a 150ms mobile client it can dominate. Rule of thumb: connection reuse $\geq 90\%$ or you’re burning latency. TFO/QUIC 0-RTT eliminate it at the cost of replay-attack considerations.

11. Common Interview Questions

1. Why three packets and not two? (Two-way can’t confirm the client can *receive*; server would trust unverified peers and old duplicate SYNs would create ghost connections.)
2. What is a SYN flood and how do SYN cookies work?
3. What options are exchanged and why can’t they change later?

12. Follow-up Questions

- “What happens if the final ACK is lost?” → server retransmits SYN-ACK; if client already sent data, that data’s ACK completes it implicitly.
- “What’s TCP Fast Open and its risk?” → data on SYN via cookie; replayable → only safe for idempotent requests.
- “Simultaneous open?” → both send SYN, both reply SYN-ACK, legal but rare (4 packets).

13. Debugging Scenarios

- `ss -tn state syn-sent` piling up → peer unreachable or SYNs dropped (firewall).
- `nstat -az TcpExtListenOverflows` rising → accept queue overflow; app too slow to accept.
- SYN retransmits at 1s, 2s, 4s... (exponential) in tcpdump → classic sign of a silent drop.

14. Best Practices

- Always pool/reuse connections; monitor handshake rate as a metric.
- Enable `net.ipv4.tcp_syncookies=1` (default on modern Linux); size `somaxconn` and app backlog.

15. Practice Questions

1. Client in Sydney, server in Virginia (RTT 200ms), TLS 1.3, no reuse: how long before the first byte of response? (~3×200ms: TCP 1 RTT + TLS 1 RTT + request/response 1 RTT.)
2. Why does a load balancer health check flood create `TIME_WAIT` on the LB, not the servers? (See 2.10 — active closer.)

Topic 2.2 — Four-Way Termination

1. Why Interviewers Ask This

Termination is where real bugs live: connection leaks, `TIME_WAIT` exhaustion, data truncation on close. “Why four packets to close but three to open?” is a standard probe.

2. Core Concept

Each direction of the byte stream closes independently (**half-close**). `FIN` means “I will send no more data” — it says nothing about receiving. So: `FIN` + `ACK` for one direction, `FIN` + `ACK` for the other = 4 packets (often 3 when the `ACK` and second `FIN` are combined).

3. Internal Working

Active closer: `close()` / `shutdown(WR)` → `FIN` → `FIN_WAIT_1` → gets `ACK` → `FIN_WAIT_2` → gets peer `FIN` → sends `ACK` → `TIME_WAIT` (waits 2×MSL, 60s on Linux). Passive closer: gets `FIN` → `CLOSE_WAIT` (app must still call `close()`) → sends its `FIN` → `LAST_ACK` → gets `ACK` → `CLOSED`.

4. Packet Flow Explanation

1. A→B `FIN`, seq=m (A done sending)
2. B→A `ACK`, ack=m+1 (B may keep sending data!)
... B can still stream data to A here (half-close) ...
3. B→A `FIN`, seq=n (B done too)
4. A→B `ACK`, ack=n+1 (A enters `TIME_WAIT` for 2×MSL)

An `RST` is the ungraceful alternative: instant teardown, any buffered data is destroyed, the peer’s next read gets `ECONNRESET`.

5. ASCII Diagram

```

A (active closer)      B (passive closer)
FIN_WAIT_1  --FIN m----->  CLOSE_WAIT  <- app must close()!
FIN_WAIT_2  <-ACK m+1-----
              <- (more data ok) ----
              <-FIN n----->  LAST_ACK
TIME_WAIT   --ACK n+1----->  CLOSED
(2*MSL)
```

6. Real Production Example

The most common leak in industry: a service stuck with thousands of sockets in `CLOSE_WAIT`. It means the *peer* closed but *your code never called close()* — typically an HTTP client whose response body wasn't drained/closed. Seen in countless Java/Go postmortems; monitored explicitly at Netflix/Uber.

7. Advantages

- Half-close enables patterns like “client sends request, shuts write, server streams full response” (classic in old HTTP and in `shutdown()`-based protocols).
- Graceful close guarantees buffered data is delivered before teardown.

8. Trade-offs

- Four packets + `TIME_WAIT` cost state and time.
- `CLOSE_WAIT` depends on application correctness — the kernel can't fix your leak.
- RST is cheap but destroys in-flight data.

9. Common Mistakes

- Believing FIN closes both directions.
- Ignoring `CLOSE_WAIT` growth (it's *always* an app bug, never a network issue).
- Calling `close()` with unread data in the receive buffer → kernel sends RST, peer may lose the response it thought was delivered.

10. Performance Impact

Termination itself is cheap; the cost is state: `TIME_WAIT` sockets (~0.2–0.5KB each) and, for clients, ephemeral-port consumption (see 2.10).

11. Common Interview Questions

1. Why 4 packets to close, 3 to open? (Open piggybacks SYN-ACK; close usually can't piggyback because the passive side may still have data to send — the FIN must wait for the app.)
2. FIN vs RST?
3. What does `CLOSE_WAIT` accumulation tell you?

12. Follow-up Questions

- “When is it 3 packets?” → passive side has nothing more to send, kernel combines ACK+FIN.
- “What happens to data written after receiving FIN?” → totally legal; FIN only closed the *other* direction.

13. Debugging Scenarios

- `ss -tn state close-wait | wc -l` growing → find the leaking client code (unclosed response bodies, missing defer/finally).
- Peer reports truncated responses → check if you close with unread request data (triggers RST that kills your queued response).

14. Best Practices

- Always drain and close HTTP response bodies; use linters (Go's `bodyclose`).
- Prefer server-side idle timeouts slightly *shorter* than client keep-alive idle timeout... actually the safe rule: whoever times out first should be the client, or the server must tolerate races (see keep-alive topic 2.9).

15. Practice Questions

1. Service A calls B via keep-alive pool. B's idle timeout is 5s, A's is 10s. Explain the race that causes sporadic `ECONNRESET` on A, and two fixes. (B closes while A sends; fixes: A idle timeout < B's, retry idempotent requests on stale-connection errors.)
2. Draw the state machines of both sides when the final ACK (packet 4) is lost. (B retransmits FIN; A in `TIME_WAIT` re-ACKs — a key reason `TIME_WAIT` exists.)

Topic 2.3 — Flow Control

1. Why Interviewers Ask This

Flow control vs congestion control confusion is the #1 TCP interview trap. Also: zero-window stalls are a real production failure mode interviewers love (“your consumer stopped reading — what happens to the producer?”).

2. Core Concept

Flow control protects the **receiver**: the receiver advertises `rwnd` (free space in its receive buffer) in every ACK; the sender may have at most `min(cwnd, rwnd)` unACKed bytes in flight. Congestion control (`cwnd`) protects the **network**. Two independent brakes; the tighter one wins.

3. Internal Working

- Receive buffer fills when the app reads slower than data arrives; advertised window shrinks toward 0.
- At `rwnd=0` the sender stops and starts **zero-window probes** (persist timer, exponential backoff) so a lost window-update can't deadlock the connection.
- **Window scaling** (option in SYN): raw 16-bit field maxes at 64KB; scale factor up to 2^{14} allows ~1GB windows — required for any high-BDP path.

4. Packet Flow Explanation

```
S→R 4KB data
R→S ACK, win=16K      (app reading fine)
S→R 16KB data
R→S ACK, win=0        (app stalled; buffer full)
S:  persist timer... probe (1 byte) → R: win=0 ... repeat
R app reads →
R→S window update, win=32K
S resumes
```

5. ASCII Diagram

```
Receiver buffer: [#####.....] free space = advertised rwnd
                  ^ app read() drains      ^ network fills
Sender: in-flight <= min(cwnd, rwnd)
rwnd=0 => sender paused, persist probes every 5s,10s,20s...
```

6. Real Production Example

Kafka/queue consumers that stall (GC, slow disk) cause producers' sends to block — backpressure propagates through TCP automatically. gRPC and HTTP/2 re-implement per-stream flow control *on top of* TCP because many streams share one TCP window (a favorite senior follow-up).

7. Advantages

- Free, automatic, per-connection backpressure — no application code needed.
- Prevents fast senders from OOMing slow receivers.

8. Trade-offs

- Head-of-line blocking when multiplexing: one slow consumer stream can throttle the shared TCP connection (HTTP/2's problem, QUIC's motivation).
- Oversized buffers hide backpressure and add latency (bufferbloat at the host level).

9. Common Mistakes

- Conflating rwnd (receiver) with cwnd (network).
- Forgetting window scaling exists → “TCP max window is 64KB” is wrong post-1992.
- Not recognizing zero-window in packet captures during “mystery stalls.”

10. Performance Impact

Max throughput $\leq \text{rwnd} / \text{RTT}$. 64KB window on 100ms RTT = 5.2 Mbps ceiling regardless of link speed. This calculation is asked verbatim in interviews.

11. Common Interview Questions

1. Flow control vs congestion control — who is protected by each?
2. Receiver app stops reading: describe the exact packet exchange that follows.
3. How does TCP achieve windows > 64KB?

12. Follow-up Questions

- “Why does HTTP/2 need its own flow control?” → per-stream fairness within one TCP window.
- “What is the persist timer preventing?” → deadlock when the window-update ACK is lost (ACKs aren't retransmitted).

13. Debugging Scenarios

- `ss -tnpi`: huge `send-q` on sender + tcpdump shows `win 0` from peer → receiver app is the bottleneck, not the network.
- Wireshark flags “TCP ZeroWindow” and “TCP Window Full” — instant diagnosis.

14. Best Practices

- Let Linux autotune buffers (`tcp_rmem` / `tcp_wmem`); don't hardcode small `SO_RCVBUF` (it disables autotuning).
- Alert on zero-window events for critical paths; they always mean “slow consumer.”

15. Practice Questions

1. 10 Gbps link, 80ms RTT: minimum window for full utilization? ($BDP = 10e9/8 \times 0.08 = 100MB$.)
2. Producer blocks on `write()` though the consumer host is idle. Consumer's `recv-q` is full. Where is the bug? (Consumer app not reading — thread pool/lock issue.)

Topic 2.4 — Congestion Control

1. Why Interviewers Ask This

It's the deepest standard TCP topic; Google interviews specifically probe BBR (they built it). It powers questions about tail latency, incast, and throughput on lossy links.

2. Core Concept

The sender maintains **cwnd** — an estimate of how much data the *network path* can absorb — and adapts it using feedback: packet loss (Reno/CUBIC), delay (Vegas/BBR), or explicit marks (ECN/DCTCP). In-flight $\leq \min(cwnd, rwnd)$.

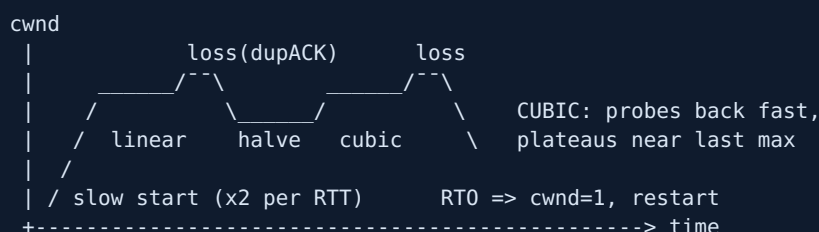
3. Internal Working

Loss-based (CUBIC, Linux default): grow cwnd until loss; on loss, multiplicative decrease (CUBIC $\times 0.7$), then grow along a cubic curve back toward the previous maximum. BBR instead measures **bottleneck bandwidth** and **min RTT** directly and paces at $BW \times RTT$, refusing to fill queues — loss is not treated as a primary signal.

4. Packet Flow Explanation

Phases of a connection (Reno-family mental model): 1. **Slow start**: cwnd doubles per RTT (exponential) from $IW=10$. 2. Cross `ssthresh` → **congestion avoidance**: +1 MSS per RTT (linear). 3. **3 dup ACKs** → fast retransmit + fast recovery: cwnd halves, no slow start. 4. **RTO timeout** → cwnd=1 MSS, back to slow start (catastrophic, ~200ms+ stall).

5. ASCII Diagram



6. Real Production Example

Google deployed **BBR** on google.com and YouTube: ~4% higher throughput globally, much better on lossy mobile paths, and lower queueing delay. Datacenters (Meta, Google) use **DCTCP/ECN** for microsecond-scale queue control. Netflix tunes CUBIC + pacing on FreeBSD for OCA appliances.

7. Advantages

- Prevents congestion collapse (the 1986 Internet meltdown that motivated all this).
- Fairness-ish sharing between flows with no coordination.

8. Trade-offs

- Loss-based CC misreads wireless/random loss as congestion → severe underutilization on lossy links (the classic $\text{throughput} \approx \text{MSS}/(\text{RTT} \times \sqrt{\text{loss}})$ Mathis formula).
- BBR (v1) can starve loss-based flows when sharing a queue; BBRv2/v3 address fairness.
- Deep buffers + loss-based CC = bufferbloat (high latency under load).

9. Common Mistakes

- Mixing up cwnd and rwnd (again — interviewers set this trap deliberately).
- Saying “loss always means congestion” without the wireless caveat.
- Not knowing the difference in severity: dup-ACK recovery (mild) vs RTO (disaster).

10. Performance Impact

Mathis formula: $\text{throughput} \leq (\text{MSS}/\text{RTT}) \cdot (1/\sqrt{p})$. At RTT 100ms, loss 1%, MSS 1448: ≈ 1.16 Mbps — on a 1 Gbps link! One number worth memorizing for interviews. Tail latency of RPCs is often dominated by rare RTO events (p99.9).

11. Common Interview Questions

1. Walk cwnd through the lifetime of a connection with one lost packet.
2. CUBIC vs BBR — signals, behavior, when each wins.
3. Why is packet loss so devastating to throughput on high-RTT links?

12. Follow-up Questions

- “What is TCP incast?” → many servers respond simultaneously to one requester (fan-in), overflow the ToR switch buffer, synchronized RTOs; fixes: DCTCP/ECN, jittered responses, larger buffers, smaller RTomin.
- “What is ECN?” → routers mark instead of drop; sender reduces without loss.
- “Why does BBR need pacing?” → sending $\text{BW} \times \text{RTT}$ in bursts would still overflow queues; pacing spreads packets at the measured rate.

13. Debugging Scenarios

- `ss -tni` shows `cwnd:` collapsing + `retrans:` climbing → path congestion/loss; correlate with `nstat TcpRetransSegs`.
- Throughput fine for small transfers, poor for bulk on cross-region links → check loss rate; consider BBR (`sysctl net.ipv4.tcp_congestion_control=bbr`).

14. Best Practices

- Enable BBR + fq pacing for internet-facing egress; DCTCP/ECN inside datacenters you control end-to-end.
- Keep RPC responses small enough to finish in slow start's first RTTs.
- Watch retransmit rate (>0.1% intra-DC deserves investigation).

15. Practice Questions

1. IW=10, MSS=1448, RTT=50ms, no loss: how long to deliver 1MB? (cwnd 10,20,40,80,160,320... cumulative segments 10,30,70,150,310,630,724 → ~7 RTTs ≈ 350ms.)
2. Why do synchronized cron jobs across 500 servers cause periodic latency spikes on the shared uplink, and name two mitigations. (Synchronized bursts → queue overflow/global loss sync; jitter the crons, ECN/AQM like fq_codel.)

Topic 2.5 — Sliding Window

1. Why Interviewers Ask This

It's the mechanism unifying reliability, ordering, flow, and congestion control — interviewers use it to test whether your TCP knowledge is a coherent model or memorized trivia. Also reappears in system design (any custom reliable protocol, Kafka in-flight batches, gRPC windows).

2. Core Concept

The sender keeps a window of bytes that may be in flight simultaneously (pipelining instead of stop-and-wait). As ACKs arrive for the oldest bytes, the window *slides* forward, admitting new bytes. Window size = $\min(\text{cwnd}, \text{rwnd})$.

3. Internal Working

Sender tracks: **SND.UNA** (oldest unACKed byte), **SND.NXT** (next byte to send). Bytes between them are in flight; bytes beyond **SND.UNA + window** are forbidden. Receiver tracks **RCV.NXT** and buffers out-of-order segments (with SACK it tells the sender exactly which ranges arrived, so only gaps are retransmitted).

4. Packet Flow Explanation

```

window = 4 segments (for illustration)
send #1 #2 #3 #4      | window [1..4] full, sender blocked
ACK 2 (got #1)        | slide -> may send #5
send #5               | in flight: 2,3,4,5
ACK 5 (cumulative: 2,3,4) | slide 3 -> send #6 #7 #8

```

Cumulative ACKs mean one ACK can confirm many segments; delayed ACKs (receiver ACKs every 2nd segment or after 40ms) reduce ACK traffic.

5. ASCII Diagram

```
byte stream: [ ACKed | in-flight (window) | may not send yet ]
               ^SND.UNA                      ^SND.UNA+min(cwnd,rwnd)
ACK arrives -> left edge advances -> right edge admits new bytes
SACK: receiver reports holes: "have 1000-1999 and 3000-3999"
      -> sender retransmits only 2000-2999
```

6. Real Production Example

Every high-throughput system reinvents this: Kafka producer `max.in.flight.requests`, gRPC/HTTP2 stream windows, TCP itself. Interviewers often ask you to design “reliable transfer over UDP” — the expected answer *is* a sliding window with SACK-like feedback (see Module 3).

7. Advantages

- Keeps the pipe full: throughput = window/RTT instead of 1 segment/RTT.
- SACK makes recovery surgical instead of go-back-N.

8. Trade-offs

- Buffering: sender must retain the whole window for possible retransmit; receiver must buffer out-of-order data.
- Ordering guarantee creates head-of-line blocking: one lost byte blocks delivery of everything after it (application sees nothing until the hole fills).

9. Common Mistakes

- Describing TCP as stop-and-wait, or as go-back-N (with SACK it's selective repeat, effectively).
- Forgetting the window is in *bytes*, not packets.
- Ignoring delayed ACKs when reasoning about packet traces (why is the receiver ACKing every other segment?).

10. Performance Impact

Throughput = window/RTT — the master equation. Also explains HoL blocking cost: on 1% loss, HTTP/2 over TCP can underperform HTTP/1.1-with-6-connections, because a single loss stalls *all* multiplexed streams (this measured result is the founding argument for QUIC).

11. Common Interview Questions

1. How does TCP achieve reliability *and* high throughput at once?
2. What is SACK and what problem does it solve?
3. Explain head-of-line blocking at the TCP level.

12. Follow-up Questions

- “One ACK is lost — does the sender retransmit?” → usually no; later cumulative ACK covers it.
- “How big should the window be?” → BDP = bandwidth × RTT.

13. Debugging Scenarios

- Wireshark “TCP Previous segment not captured” + receiver ACKing same value repeatedly (dup ACKs) → loss + window stalled at the hole.
- Throughput plateaus below link speed with zero loss → window limit; check `ss -tni` (rwnd or cwnd bound? `wscale` present?).

14. Best Practices

- Ensure window scaling + SACK enabled (defaults; but middleboxes have historically stripped them — check SYN options in captures).
- For custom protocols: always design with sliding window + selective ACK, not stop-and-wait.

15. Practice Questions

1. RTT 60ms, want 1 Gbps: window? ($1e9/8 \times 0.06 = 7.5\text{MB.}$)
2. Segments 1–10 sent; #4 lost. With SACK, exactly which packets flow next? (Receiver dup-ACKs 4 with SACK blocks 5–10; sender fast-retransmits only #4; on its arrival receiver ACKs 11.)

Topic 2.6 — Retransmission

1. Why Interviewers Ask This

Retransmission behavior explains most “mystery latency spikes” in production. Interviewers test whether you know both mechanisms (RTO vs fast retransmit) and their radically different costs.

2. Core Concept

Two triggers: **RTO** (retransmission timeout — timer expiry, catastrophic: cwnd resets to 1) and **fast retransmit** (3 duplicate ACKs — mild: cwnd halves, pipe keeps flowing). Modern additions: **TLP** (tail loss probe) rescues losses at the end of a burst where no dup ACKs can be generated, and **RACK** uses time-based ordering instead of counting dup ACKs.

3. Internal Working

RTO computed from smoothed RTT: `SRTT + 4 × RTTVAR` (Jacobson/Karels), Linux min 200ms, initial 1s. Each unsuccessful retransmit doubles RTO (exponential backoff: 1s, 2s, 4s... up to ~15 retries / ~13–30 min before the connection dies). Karn’s algorithm: RTT samples from retransmitted segments are ignored (ambiguous ACKs).

4. Packet Flow Explanation

Fast retransmit:

```
send seg 1..6, seg2 lost
recv: ACK2, (3 arrives)ACK2+SACK3, (4)ACK2+SACK3-4, (5)ACK2+SACK3-5
sender: 3rd dup ACK => retransmit seg2 immediately, cwnd/=2
recv seg2 => ACK7 (everything filled) => window jumps forward
```

Tail case: only seg6 (the last) lost → no following packets → no dup ACKs → without TLP you wait a full RTO. TLP sends a probe after $\sim 2 \times \text{RTT}$ to trigger SACK feedback.

5. ASCII Diagram

```

RTO path:      |--send--X.....(200ms+ silence).....retransmit|
               cwnd -> 1, slow start again                (p99 latency spike)
FastRetx path: |--send--X-3 dupACKs(==1 RTT)-retransmit-|
               cwnd -> cwnd/2, keeps streaming            (small blip)

```

6. Real Production Example

The classic “p99 = 200ms exactly” signature: your median is 2ms but p99 sits at ~200ms → Linux RTOMin. Common with short request/response flows where the response is 1–2 packets (no dup ACKs possible). Seen and documented at Google (motivating TLP/RACK, both authored by Google engineers).

7. Advantages

- Reliability over arbitrary lossy networks with zero app involvement.
- Backoff prevents retransmission storms from melting congested networks.

8. Trade-offs

- RTO min 200ms is an eternity for 1ms-RTT datacenter RPCs.
- Retransmits waste bandwidth if loss was actually reordering (RACK largely fixes this).
- TCP retries invisibly — your app-level timeout may fire while TCP is still dutifully retrying underneath.

9. Common Mistakes

- Thinking every loss costs an RTO — fast retransmit handles most mid-stream loss in ~1 RTT.
- Setting app timeouts without knowing TCP’s schedule (app retry at 100ms + TCP RTO at 200ms = duplicate work).
- Ignoring that reordering can fake dup ACKs (spurious retransmits).

10. Performance Impact

Retransmit rate is *the* canonical network health metric. Intra-DC target: <0.01%. Each RTO ≈ +200ms tail latency. `nstat : TcpRetransSegs` , `TcpExtTCPLostRetransmit` , `TcpExtTCPTimeouts` (timeouts are the expensive kind).

11. Common Interview Questions

1. RTO vs fast retransmit — triggers and cost difference.
2. How is RTO calculated? Why not fixed?
3. Why do tail losses hurt more than mid-stream losses?

12. Follow-up Questions

- “Why min RTO 200ms when DC RTT is 0.1ms?” → protection against spurious retransmits with delayed ACKs (40ms); datacenter stacks lower it or use RACK/TLP.
- “What’s Karn’s algorithm?” → don’t sample RTT from retransmitted segments.

13. Debugging Scenarios

- Latency histogram with a spike at exactly ~200ms/1s → RTO events; count `TcpExtTCPTimeouts` .

- `ss -tni` shows `retrans:X/Y` per connection — live view of who's suffering.
- Wireshark: “TCP Retransmission” vs “TCP Fast Retransmission” vs “TCP Spurious Retransmission” labels tell you the mechanism.

14. Best Practices

- Keep RPC payloads multi-packet or enable TLP/RACK (default in modern kernels) to avoid tail-loss RTOs.
- Align app-level timeouts/retries with TCP behavior: hedge requests after ~p95 rather than blind fast retries.

15. Practice Questions

1. A single-packet response is lost. With and without TLP, when is it recovered? (Without: RTO $\geq 200\text{ms}$. With: probe at $\sim 2 \times \text{RTT}$, then SACK-driven repair.)
2. Your service's p50=3ms, p99=204ms, p99.9=1.2s. Explain all three numbers. (Base, +RTOmin event, +RTO backoff 200ms $\rightarrow$ 400ms... or 1s initial RTO on new connections.)

Topic 2.7 — Slow Start

1. Why Interviewers Ask This

Slow start explains why “first request is slow,” why CDNs put servers near users, and why connection reuse matters — all favorite system-design threads.

2. Core Concept

New connections don't know the path capacity, so `cwnd` starts at `IW=10` segments ($\sim 14.6\text{KB}$) and **doubles every RTT** (exponential — the name “slow” is historical, vs. dumping the whole window at once) until loss or `ssthresh`.

3. Internal Working

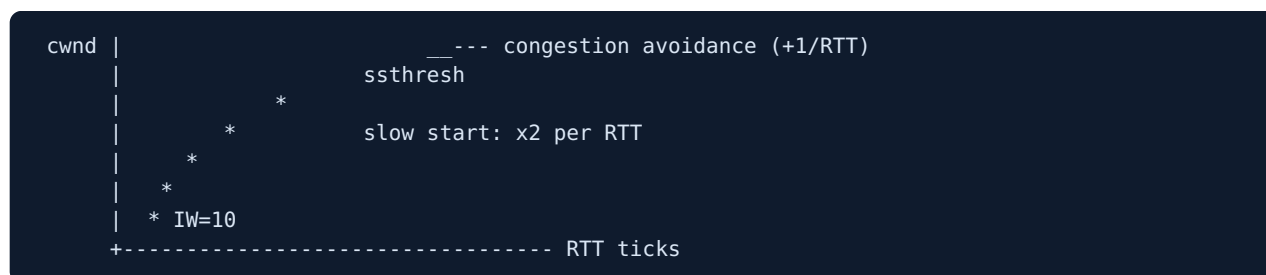
For every ACK received, `cwnd` ± 1 MSS $\rightarrow$ per-RTT doubling. Crossing `ssthresh` switches to congestion avoidance ($+1$ MSS/RTT). After an RTO, `ssthresh` = `cwnd/2` and `cwnd` restarts at 1. **Slow-start restart:** after an idle period, `cwnd` collapses back toward `IW` (tunable `tcp_slow_start_after_idle` — disable it for keep-alive RPC pools, a great production detail to mention).

4. Packet Flow Explanation

RTT-by-RTT for a fresh connection, MSS 1448:

```
RTT1: send 10 segs (14.5KB)   RTT4: 80 segs (116KB)
RTT2: send 20 segs (29KB)    RTT5: 160 segs (232KB)
RTT3: send 40 segs (58KB)    RTT6: 320 segs (463KB)
1MB file needs ~7 RTTs of ramp-up regardless of link speed.
```

5. ASCII Diagram



6. Real Production Example

This is *the* argument for CDNs beyond caching: TLS + slow start from a 10ms-away edge ramps ~15x faster than from a 150ms-away origin. It's also why HTTP/2 consolidates onto one connection — a warm connection with large cwnd beats six cold ones.

7. Advantages

- Probes unknown paths safely; prevents instant queue overflow from new flows.

8. Trade-offs

- Short transfers (most web/API traffic!) finish before reaching link capacity — they live entirely inside slow start.
- After idle, pools silently lose their warmed cwnd unless tuned.

9. Common Mistakes

- “Slow start means TCP is slow at first, linearly” — it's exponential.
- Forgetting IW=10 (old texts say 1–4; RFC 6928 raised it).
- Not knowing slow-start-after-idle exists (your “persistent” connection re-ramps anyway!).

10. Performance Impact

Time to fetch N bytes $\approx \text{RTT} \times \log_2(N/\text{IW} \cdot \text{MSS}) + N/\text{bandwidth}$. For small objects, $\text{RTT} \times \log$ term dominates → latency is RTT-bound, not bandwidth-bound. Core web-performance math.

11. Common Interview Questions

1. Why is the first request on a connection slower even after the handshake?
2. How many RTTs to transfer 1MB on a fresh connection?
3. Why do CDNs help even for dynamic (uncacheable) content? (Terminate TLS+TCP near user; warmed backbone connections to origin.)

12. Follow-up Questions

- “What resets cwnd besides loss?” → RTO, idle restart.
- “How does BBR handle startup?” → similar exponential probing (STARTUP phase) but exits on measured BW plateau, not loss.

13. Debugging Scenarios

- “First call after deploy is slow, then fast” → cold connections + slow start (plus app JIT/caches; distinguish by `ss -tni` cwnd growth).
- Batch job over long-RTT link never reaches expected throughput → transfer shorter than ramp time; use parallel streams or persistent warmed connections.

14. Best Practices

- Reuse connections aggressively; disable `tcp_slow_start_after_idle` for internal RPC fleets.
- Keep critical first responses within initial cwnd (~14KB).
- Terminate TLS near users (edge/PoP).

15. Practice Questions

1. RTT 100ms, 500KB page, fresh connection: rough download time? (~6 doubling RTTs + transfer ≈ 600–700ms even on gigabit.)
2. Two designs: 6 parallel cold connections vs 1 warm connection with cwnd=200. Which delivers 300KB faster on RTT 50ms and why?

Topic 2.8 — Fast Recovery

1. Why Interviewers Ask This

It completes the congestion-control story and distinguishes candidates who understand *why* TCP stays fast despite loss. “What happens on 3 dup ACKs” is a standard senior checkpoint.

2. Core Concept

On 3 duplicate ACKs (loss inferred but data still flowing → network isn’t collapsed), TCP retransmits the missing segment and **halves** cwnd instead of resetting to 1 — skipping slow start. Recovery in ~1 RTT.

3. Internal Working

Classic NewReno: $ssthresh = cwnd/2$; $cwnd = ssthresh (+3 \text{ for the dup ACKs})$; each additional dup ACK inflates cwnd by 1 (packets have left the network — “packet conservation”); on the ACK covering new data, cwnd deflates to ssthresh and normal avoidance resumes. Linux uses **PRR** (Proportional Rate Reduction): paces the reduction smoothly across the recovery RTT — avoids both bursts and stalls.

4. Packet Flow Explanation

```
cwnd=20. Segments 1..20 in flight, #5 lost.
dupACK(5) x3 arrive      -> retransmit #5; ssthresh=10
during recovery: SACK shows 6..20 delivered; PRR sends ~1 new
                        segment per 2 ACKs, converging cwnd to 10
ACK 21 arrives           -> recovery ends; cwnd=10; +1/RTT resumes
Total damage: throughput halved for a while; NO 200ms stall.
```


5. ASCII Diagram

```

cwnd 20 ----\
               \ fast retransmit + PRR
                 \_____ 10 _____ +1/RTT ____/
vs RT0: 20 --| 200ms silence |-- 1 -- slow start --
(fast recovery = dip; RT0 = cliff)

```

6. Real Production Example

On a lossy mobile network at 1% loss, a video stream survives almost entirely on fast recovery — throughput degrades gracefully. When monitoring streaming fleets (Netflix), the ratio `TCPFastRetrans : TCPTimeouts` is a health indicator: timeouts trending up means losses are happening in places dup ACKs can't signal (tails, bursts, total outage).

7. Advantages

- Keeps the pipe flowing during isolated loss; ~1 RTT repair.
- With SACK+PRR, handles multiple losses per window cleanly (classic Reno struggled: one RTT per lost segment).

8. Trade-offs

- Needs enough in-flight data after the hole to generate 3 dup ACKs — useless for tiny transfers and tail losses (hence TLP/RACK).
- Packet reordering > dup-ACK threshold triggers spurious retransmits + unnecessary cwnd cuts (RACK's time-based detection mitigates).

9. Common Mistakes

- Confusing fast retransmit (the resend trigger) with fast recovery (the cwnd management that follows).
- Claiming cwnd goes to 1 on any loss.
- Forgetting SACK's role — modern recovery is SACK-driven, not dup-ACK-counting.

10. Performance Impact

Single loss with fast recovery $\approx$ throughput dip of ~50% for one congestion epoch, no latency cliff. Single loss requiring $RTO \approx +200\text{ms}$ latency and $cwnd=1$. For p99 tuning you care almost exclusively about eliminating the RTO class.

11. Common Interview Questions

1. Exactly what happens on the third duplicate ACK?
2. Why is dup-ACK loss “better” than timeout loss?
3. How do SACK and fast recovery interact?

12. Follow-up Questions

- “Why inflate cwnd during recovery?” → each dup ACK proves a packet left the network; conservation allows sending replacements.
- “What if the retransmission itself is lost?” → RTO (or RACK-based re-retransmit in modern stacks).

13. Debugging Scenarios

- `nstat`: healthy loss shows `TcpExtTCPFastRetrans` rising with few `TCPTimeouts`; the reverse means tail-loss or blackholes.
- Spurious retransmits + reordering: `TcpExtTCPDSACKRecv` (peer says “you sent me a duplicate”) → suspect multipath reordering, not real loss.

14. Best Practices

- Keep modern defaults: SACK, RACK, TLP, PRR (Linux ≥4.18 has them all on).
- Avoid per-packet load balancing (ECMP on flows, not packets) to prevent reordering that defeats recovery.

15. Practice Questions

1. `cwnd=32`, one segment lost mid-window, SACK on. Estimate delivery delay added for the lost segment. (~1 RTT for 3 dup ACKs + retransmit flight.)
2. Why might a request/response service with 4KB responses see RTOs but a bulk-transfer service on the same path see none? (4KB = 3 segments; a tail loss can't collect 3 dup ACKs.)

Topic 2.9 — Keep-Alive

1. Why Interviewers Ask This

Two distinct mechanisms share the name — TCP keep-alive vs HTTP keep-alive — and interviewers deliberately test the distinction. Idle-connection death via NAT/firewall/LB timeouts is a top-5 production issue.

2. Core Concept

TCP keep-alive: kernel-level probe packets on an idle connection to detect dead peers and keep middlebox state alive. Defaults (Linux): first probe after **7200s**, then 9 probes 75s apart — uselessly long for most middleboxes; must be tuned. **HTTP keep-alive**: application-level connection reuse for multiple requests (persistent connections, Module 4).

3. Internal Working

Keep-alive probe = segment with `seq = snd.nxt-1` and no/garbage payload; a healthy peer replies ACK. No reply after N probes → connection torn down, app gets ETIMEDOUT. Configurable per-socket: `TCP_KEEPIDLE`, `TCP_KEEPINTVL`, `TCP_KEEPCNT`.

4. Packet Flow Explanation

The failure it prevents: NAT/LB drops idle-flow state after e.g. 350s (AWS NLB default; typical NAT ~300s, AWS Lambda/NAT GW 350s, Azure LB 4 min):

```
t=0      last real packet
t=350s   NAT expires flow entry silently
t=400s   client sends request -> NAT: unknown flow -> drop (or RST)
          client TCP retransmits into the void... hangs until timeout
With keep-alive every 60s: NAT entry stays fresh; problem never occurs.
```

5. ASCII Diagram

```
app idle...[KA probe]-> <-ACK ....[KA probe]-> <-ACK ....
                (every KEEPINTVL after KEEPIDLE)
dead peer:  [KA]->x  [KA]->x  ... x KEEPINT => ETIMEDOUT to app
middlebox:  each probe refreshes NAT/conntrack/LB idle timers
```

6. Real Production Example

Database connection pools through cloud load balancers: the canonical incident is “first query after quiet period fails/hangs.” AWS documents setting TCP keep-alive < 350s for NLB. PostgreSQL, gRPC (HTTP/2 PING frames — an *application-layer* keep-alive), and Kubernetes clients all ship keep-alive knobs because of this exact class of outage.

7. Advantages

- Detects dead peers (crashed host, unplugged cable) that would otherwise leave sockets hanging forever (a silent peer crash produces *no* packet).
- Keeps stateful middleboxes from expiring live-but-idle flows.

8. Trade-offs

- Probes cost (tiny) bandwidth and wake radios on mobile (battery).
- Kernel keep-alive says “peer TCP is alive,” not “application is healthy” — a deadlocked app still ACKs. gRPC’s HTTP/2 PING checks one layer higher; app-level health checks higher still.

9. Common Mistakes

- Conflating TCP keep-alive with HTTP keep-alive in answers — instant signal of shallow knowledge.
- Relying on Linux defaults (2 hours!) and wondering why NAT killed the pool.
- Believing keep-alive detects application hangs.

10. Performance Impact

Negligible traffic; the *absence* of keep-alive costs you: hung requests for minutes (retransmit backoff into a dead flow), pool exhaustion from zombie connections, and thundering reconnect herds after LB failovers.

11. Common Interview Questions

1. TCP keep-alive vs HTTP keep-alive?
2. Why do idle DB connections through a NAT/LB die, and how do you fix it?
3. How does a client detect a *crashed* (not closed) server? (No FIN was ever sent — only KA probes or a write → RST reveal it.)

12. Follow-up Questions

- “Why doesn’t the client notice immediately when the server host dies?” → TCP is silent when idle; no packets = no information.
- “Where would you use app-level pings instead?” → gRPC/HTTP2 PING, WebSocket ping/pong — they verify the app event loop, traverse L7 proxies meaningfully, and work per-stream.

13. Debugging Scenarios

- “First request after idle hangs then times out; retry works” → middlebox idle-timeout; set KA < middlebox timeout.
- Pool full of ESTABLISHED sockets to a rebooted server → no keep-alive; zombies until first write fails.

14. Best Practices

- Set TCP keep-alive (or app pings) to $\sim\frac{1}{3}$ of the smallest middlebox idle timeout on the path (e.g., 60s).
- Prefer application-level pings for L7-proxied protocols (gRPC keepalive time/timeout).
- Also set client-side pool max-idle-age below server/LB idle timeout to avoid the close race.

15. Practice Questions

1. Path: client → NAT (300s idle) → NLB (350s) → server. Choose keep-alive parameters. (KEEPIDLE≈90s, KEEPINTVL≈30s, KEEP CNT≈3 — probes well under 300s.)
2. gRPC streams over an idle-timeout L7 proxy keep dying at exactly 60s. TCP keep-alive is on. Why doesn't it help? (Proxy counts L7 activity; TCP probes don't create HTTP/2 frames — need HTTP/2 PING.)

Topic 2.10 — TIME_WAIT

1. Why Interviewers Ask This

TIME_WAIT is the most-misunderstood TCP state and a genuine scaling bottleneck for proxies and load-test clients. “Server has 50k TIME_WAIT sockets — is that a problem?” is a classic senior question with a nuanced answer.

2. Core Concept

The side that closes **first** (active closer) holds the socket in TIME_WAIT for **2×MSL** (Linux: fixed 60s) after the final ACK. Purpose: (1) if that final ACK is lost, the peer retransmits its FIN and someone must be there to re-ACK; (2) let stray delayed segments from this connection die before the 4-tuple (src IP, src port, dst IP, dst port) can be reused — otherwise an old segment could corrupt a new connection with matching sequence numbers.

3. Internal Working

TIME_WAIT sockets are lightweight kernel records (a few hundred bytes, no buffers). The real constraint is **client-side**: each outbound connection to the same (dst IP, dst port) burns one ephemeral port (~28k–64k range, `ip_local_port_range`) for 60s → max sustainable NEW connection rate to one destination $\approx 28,000/60 \approx \sim 470$ **conn/s**. Mitigations: `tcp_tw_reuse` (safe, timestamp-based, outbound only), more source IPs, wider port range, and above all *connection reuse*. (`tcp_tw_recycle` was broken with NAT and removed from Linux.)

4. Packet Flow Explanation

```
A closes first:
A: FIN ->      B: ACK, FIN ->      A: final ACK, enter TIME_WAIT(60s)
case lost final ACK: B retransmits FIN -> A (in TIME_WAIT) re-ACKs. Saved.
case no TIME_WAIT: B's FIN hits closed port -> RST; B sees error on a
                  connection that closed "successfully".
case reuse too soon: old delayed segment (same 4-tuple, in-window seq)
                  -> data corruption on the NEW connection.
```

5. ASCII Diagram

```
active closer: ESTAB -> FIN_WAIT_1 -> FIN_WAIT_2 -> TIME_WAIT --60s--> gone
passive closer: ESTAB -> CLOSE_WAIT -> LAST_ACK -> CLOSED
WHO CLOSES FIRST PAYS THE TIME WAIT TAX.
Proxy fleets: make the *client* (or the side with more IPs) close first,
or better: keep-alive so you rarely close at all.
```

6. Real Production Example

Load-test clients that open a new connection per request stall at ~470 RPS per (srcIP,dstIP,dstPort) with connect() EADDRNOTAVAIL — a rite-of-passage incident. Reverse proxies (NGINX → upstream) hit the same wall; standard fixes in industry: upstream keep-alive pools, multiple upstream ports/IPs, `tcp_tw_reuse=1`.

7. Advantages

- Correctness: protects the close handshake and prevents old-segment corruption of new connections. It's a feature, not a leak.

8. Trade-offs

- Ephemeral-port pressure on busy clients/proxies.
- 60s of table entries (memory is minor; port space is the issue).
- Asymmetric: you can shift the cost by choosing who closes first, but not eliminate it.

9. Common Mistakes

- "TIME_WAIT means connection leak — let's kill it" → it's normal; the fix is reuse, not eradication.
- Enabling the (removed/dangerous) `tcp_tw_recycle` advice from old blogs — breaks clients behind NAT.
- Thinking TIME_WAIT happens on the passive closer, or on the server always ("server" vs "active closer" confusion).

10. Performance Impact

Server with 100k TIME_WAIT inbound sockets: ~tens of MB RAM, harmless (different client IPs → no port conflict). Client/proxy with TIME_WAIT to one upstream: hard cap ≈ port_range/60 new conns/sec. Know both sides of this asymmetry — that's the senior answer.

11. Common Interview Questions

1. What is TIME_WAIT for? (Both reasons.)

2. 50k TIME_WAIT sockets on a box — problem or not? (Depends: inbound on server = fine; outbound to one upstream = port exhaustion risk.)
3. How do you run 10k conn/s from one proxy to one backend? (Keep-alive pools; tw_reuse; multiple local IPs/ports.)

12. Follow-up Questions

- “Why 2×MSL specifically?” → covers a segment’s maximum round trip of lingering copies (MSL each way).
- “What does SO_REUSEADDR actually do?” → lets you bind() a listener over a TIME_WAIT socket (server restart case); it does *not* bypass TIME_WAIT for outbound 4-tuples.
- “How does tcp_tw_reuse stay safe?” → TCP timestamps ensure old segments are distinguishable; applies only to outgoing connects.

13. Debugging Scenarios

- `connect(): Cannot assign requested address` under load → `ss -tan state time-wait | wc -l` + check `ip_local_port_range`; fix with pooling/tw_reuse.
- Server restart fails `bind: Address already in use` → missing SO_REUSEADDR on the listener.

14. Best Practices

- Design so connections are long-lived; TIME_WAIT churn is a symptom of connection-per-request.
- On proxies: `net.ipv4.tcp_tw_reuse=1`, widen `ip_local_port_range`, multiple upstream addresses.
- Never disable TIME_WAIT semantics globally; never use removed recycle hacks.

15. Practice Questions

1. Port range 32768–60999, TIME_WAIT 60s, one upstream ip:port. Max new conn/s? $((60999 - 32768) / 60 \approx 470.)$
2. An LB health-checks 200 backends every 2s with fresh connections, closing actively. How many TIME_WAIT sockets does it hold at steady state? $(200 \times 30 \text{ checks-in-60s} = 6,000)$ — and why per-backend port pressure is still low.)

MODULE 2 — One-Page Cheat Sheet

HANDSHAKE	SYN(x) / SYN-ACK(y,ack x+1) / ACK — 1 RTT; options only in SYN SYN q vs accept q; SYN cookies = stateless defense
CLOSE	FIN/ACK + FIN/ACK (half-close). Active closer -> TIME_WAIT 60s CLOSE_WAIT pileup = YOUR app forgot close()
FLOW CTRL	rwnd protects receiver; zero-window + persist probes
CONGESTION	cwnd protects network; in-flight $\leq \min(\text{cwnd}, \text{rwnd})$ slow start: IW=10, $x2/\text{RTT} \rightarrow \text{ssthresh} \rightarrow +1 \text{ MSS}/\text{RTT}$ 3 dupACKs: fast retransmit, $\text{cwnd}/2$, PRR (mild) RTO ($\geq 200\text{ms}$): $\text{cwnd}=1$, slow start again (p99 killer) CUBIC=loss-based (default) BBR=model-based (Google)
SLIDING WIN	throughput = window/RTT ; BDP = BW x RTT ; SACK fills holes Mathis: $\text{tput} \sim \text{MSS}/(\text{RTT} \cdot \sqrt{\text{loss}})$
RETRANSMIT	RTO = $\text{SRTT} + 4 \cdot \text{RTT}_{\text{VAR}}$, min 200ms, exp backoff; TLP/RACK fix tails
KEEP-ALIVE	TCP KA(default 2h! tune to ~60s) vs HTTP KA(reuse) vs h2 PING middlebox idle timeouts (NAT ~300s, NLB 350s) kill silent flows
TIME_WAIT	$2 \cdot \text{MSL} = 60\text{s}$; client-side port math: $\sim 28\text{k}/60\text{s} \approx 470 \text{ conn/s/dst}$ fixes: keep-alive pools, tcp_tw_reuse, more IPs. NOT recycle.
TOOLS	ss -tnpi nstat tcpdump wireshark expert info
KEY NUMBERS	IW 10 (~14KB) RTOmin 200ms TW 60s KA 7200s default

MODULE 2 — Top Interview Questions

1. Full lifecycle: draw every packet and state from connect() to close() including sequence numbers.
2. Flow control vs congestion control — who's protected, what's the signal, where do they meet? ($\min(\text{cwnd}, \text{rwnd})$).
3. One packet is lost. Walk both recovery paths and their latency cost.
4. Why is TCP slow on high-RTT lossy links? Derive it ($\text{window}/\text{RTT} + \text{Mathis}$).
5. p99 latency is exactly 200ms above p50 — explain.
6. CLOSE_WAIT vs TIME_WAIT pileups — which is a bug, whose bug, and fixes.
7. Idle connections through NAT/LB die — mechanism and three-layer fix (TCP KA, app ping, pool max-idle).
8. Design a proxy doing 10k conn/s to few upstreams — enumerate the TCP limits you'll hit (ports/ TIME_WAIT, accept queue, buffers=BDP, CC choice).
9. Why did Google build BBR? What signal replaced loss?
10. What are SYN cookies and what do they sacrifice?

MODULE 2 — Common Mistakes

- cwnd/rwnd confusion (the classic filter).
- "Loss $\Rightarrow \text{cwnd}=1$ " — only RTO does that.
- Treating TIME_WAIT as a leak; treating CLOSE_WAIT as normal.
- TCP vs HTTP keep-alive conflation; trusting the 2-hour default.
- Forgetting options (window scale/SACK) live only in the SYN.
- App retry policies ignorant of TCP's own retransmit schedule (retry storms).

- “write() returned = delivered.”

MODULE 2 — Mock Interview (15 min)

Q1. “Your cross-region replication (RTT 120ms) runs at 40 Mbps on a 10 Gbps pipe. Diagnose.” *Strong answer:* Compute implied window: $40\text{Mbps} \times 0.12\text{s} = 600\text{KB}$ — suspiciously close to a clamped buffer; check `ss -tni` (wscale? cwnd? rwnd-limited?), check loss (Mathis: 40Mbps at 120ms $\Rightarrow$ ~0.1% loss could explain it via CUBIC), fixes: raise tcp_rmem/wmem max (or stop app from setting SO_RCVBUF), consider BBR, parallel streams.

Q2. “After a deploy, clients report sporadic ECONNRESET at low traffic, mostly after quiet periods.” *Strong answer:* Idle-timeout race — server/LB closes idle keep-alive connections while client reuses them; verify timing correlation with idle age; fix client pool max-idle < server idle timeout, add retry-on-stale for idempotent methods; mention FIN vs RST observation in capture.

Q3. “Explain to a junior why we didn’t just disable TIME_WAIT.” *Strong answer:* it guards final-ACK loss and stale-segment corruption of reused 4-tuples; the pressure is ports, so pool connections and enable tw_reuse — removing the state risks silent data corruption, the worst class of bug.